

Intelligent System Development

From Deep Learning to Knowledge Graph, RAG,
and Intelligent Agents

Dinh Que Tran

Student Lecture and Practical Tutorial
Department of Information Technology

2026

Abstract

This lecture presents a unified development framework for modern intelligent systems. The course begins with Deep Learning, which enables systems to learn representations and predictive functions from data. It then introduces Knowledge Graphs for representing structured knowledge and relationships, Retrieval Augmented Generation (RAG) for retrieving external information before language generation, and intelligent agents for combining reasoning, memory, tools, retrieval, and action.

The central message is that these technologies are complementary. Deep Learning provides learned representations and predictive models; Knowledge Graphs provide structured relationships; RAG provides access to external information; large language models provide language understanding and generation; and agents coordinate these components to achieve a goal.

Students progressively implement these ideas using Python, embeddings, vector retrieval, knowledge graphs, and language models. The final project is an intelligent assistant that can retrieve information, reason over structured knowledge, use learned models, and perform useful actions.

Contents

1	Introduction	4
2	From Machine Learning to Intelligent Systems	4
3	Deep Learning as the Learning Component	5
4	Why Learned Models Need External Knowledge	6
5	Knowledge Representation	6
6	Knowledge Graphs	6
7	Example: University Knowledge Graph	7
8	Creating a Small Knowledge Graph in Python	8
9	Graph Databases	8
10	What is RAG?	8

11 Document Embeddings	9
12 Basic RAG Pipeline	9
13 A Minimal RAG Implementation	9
14 Why RAG is Important	10
14.1 Current Information	10
14.2 Domain Knowledge	10
14.3 Evidence	10
14.4 Reduced Hallucination	10
15 Knowledge Graph versus Vector RAG	10
16 Knowledge Graph + RAG	11
17 Example: University Intelligent Assistant	11
17.1 Step 1: Query the Knowledge Graph	12
17.2 Step 2: Retrieve Documents	12
17.3 Step 3: Combine Evidence	12
17.4 Step 4: Generate Answer	12
18 GraphRAG	12
19 GraphRAG Example	13
20 From RAG to Intelligent Agents	13
21 Agent Architecture	13
22 Tools	14
23 Deep Learning + KG + RAG + Agent	14
24 A Unified Mathematical View	15
25 Practical Development Roadmap	15
26 Practical Project	17
27 Example Project: University Assistant	17
28 Example User Interaction	18
29 Evaluation of an Intelligent System	18
29.1 Prediction Accuracy	19
29.2 Retrieval Quality	19
29.3 Answer Quality	19
29.4 Agent Performance	19
30 Common Failure Modes	19
30.1 Deep Learning Failure	20
30.2 Knowledge Graph Failure	20
30.3 RAG Failure	20
30.4 LLM Failure	20

30.5 Agent Failure	20
31 Implementation Environment	20
32 Suggested Student Workflow	21
33 Mini Project 1: Semantic Search	22
34 Mini Project 2: RAG Chatbot	22
35 Mini Project 3: Knowledge Graph	22
36 Final Project: Intelligent Agent	23
37 Recommended Final Report Structure	23
38 The Big Picture	24
39 Exercises	24
40 Final Message to Students	25

1 Introduction

Artificial Intelligence systems have evolved from systems that perform a single prediction into systems that can combine learning, knowledge, retrieval, reasoning, and action.

A traditional machine learning system may answer a question such as

“Is this image a cat or a dog?”

A more advanced intelligent system should be able to answer

“Why is this image classified as a cat, what related information do we know about cats, and what should the user do next?”

These are fundamentally different capabilities.

A useful development framework is

$$\boxed{\text{Intelligent System} = \text{Learning} + \text{Knowledge} + \text{Retrieval} + \text{Reasoning} + \text{Action}.} \quad (1)$$

In this framework:

- Deep Learning provides learning.
- Knowledge Graphs provide structured knowledge.
- RAG provides information retrieval.
- Large Language Models provide language understanding and generation.
- Agents provide planning, tool use, memory, and action.

The goal of this lecture is to understand how these components can be integrated into one intelligent system.

2 From Machine Learning to Intelligent Systems

Students have previously studied machine learning and deep learning models.

A typical supervised learning problem can be written as

$$y = f_{\theta}(x), \quad (2)$$

where

- x is the input,
- y is the target,
- f_{θ} is a learned model,
- θ represents trainable parameters.

For example,

$$\text{image} \longrightarrow \text{CNN} \longrightarrow \text{class}. \quad (3)$$

For time-series prediction,

$$(x_1, x_2, \dots, x_T) \longrightarrow \text{LSTM} \longrightarrow \hat{x}_{T+1}. \quad (4)$$

These models are powerful, but they normally solve a relatively specific computational problem.

An intelligent system must go beyond prediction.

It may need to:

1. understand a user query;
2. retrieve relevant information;
3. use structured knowledge;
4. call a prediction model;
5. reason about several pieces of information;
6. use external tools;
7. remember previous interactions;
8. produce an answer;
9. perform an action.

This motivates the transition

$$\text{Deep Learning} \rightarrow \text{Knowledge} \rightarrow \text{Retrieval} \rightarrow \text{Agent}. \quad (5)$$

3 Deep Learning as the Learning Component

Deep Learning learns representations from data.

A neural network can be viewed abstractly as

$$h = f_{\theta}(x), \quad (6)$$

where h is a learned representation.

For an image,

$$x_{\text{image}} \rightarrow h_{\text{visual}} \rightarrow y. \quad (7)$$

For text,

$$x_{\text{text}} \rightarrow h_{\text{text}} \rightarrow y. \quad (8)$$

For time series,

$$X_{1:T} \rightarrow h_T \rightarrow \hat{y}. \quad (9)$$

These representations become important later.

A modern intelligent system may use several learned representations:

$$\begin{aligned} h_{\text{image}} &= f_{\theta}^{\text{image}}(x), \\ h_{\text{text}} &= f_{\theta}^{\text{text}}(q), \\ h_{\text{time}} &= f_{\theta}^{\text{time}}(X). \end{aligned} \quad (10)$$

The system can then combine these representations with external knowledge.

4 Why Learned Models Need External Knowledge

Suppose a student builds a neural network for university information.

The model may learn patterns from training data, but it does not automatically know a newly published university regulation.

This illustrates an important distinction:

$$\text{Model Parameters} \neq \text{Current Knowledge.} \quad (11)$$

A trained model contains information encoded during training. An intelligent system often needs information that is:

- current,
- private,
- domain-specific,
- stored in documents,
- stored in databases,
- continuously updated.

Therefore, the system should retrieve external information when needed.

This is the basic motivation for RAG.

5 Knowledge Representation

Before introducing Knowledge Graphs, consider the difference between unstructured and structured information.

A document may contain:

“Artificial Intelligence is offered by the Department of Information Technology. The course requires Linear Algebra and Programming.”

Humans can easily identify the relationships.

A computer can represent them explicitly as

$$\begin{aligned} &(\text{Artificial Intelligence}, \text{offered_by}, \text{Information Technology}), \\ &(\text{Artificial Intelligence}, \text{requires}, \text{Linear Algebra}), \\ &(\text{Artificial Intelligence}, \text{requires}, \text{Programming}). \end{aligned} \quad (12)$$

These are knowledge triples.

6 Knowledge Graphs

Definition 6.1 (Knowledge Graph). *A Knowledge Graph is a graph-based representation in which entities are represented as nodes and semantic relationships between entities are represented as edges.*

A simple graph is

$$G = (V, E), \quad (13)$$

where V is the set of entities and E is the set of relationships.

A knowledge graph may contain triples

$$(s, r, o), \quad (14)$$

where

- s is the subject,
- r is the relation,
- o is the object.

For example,

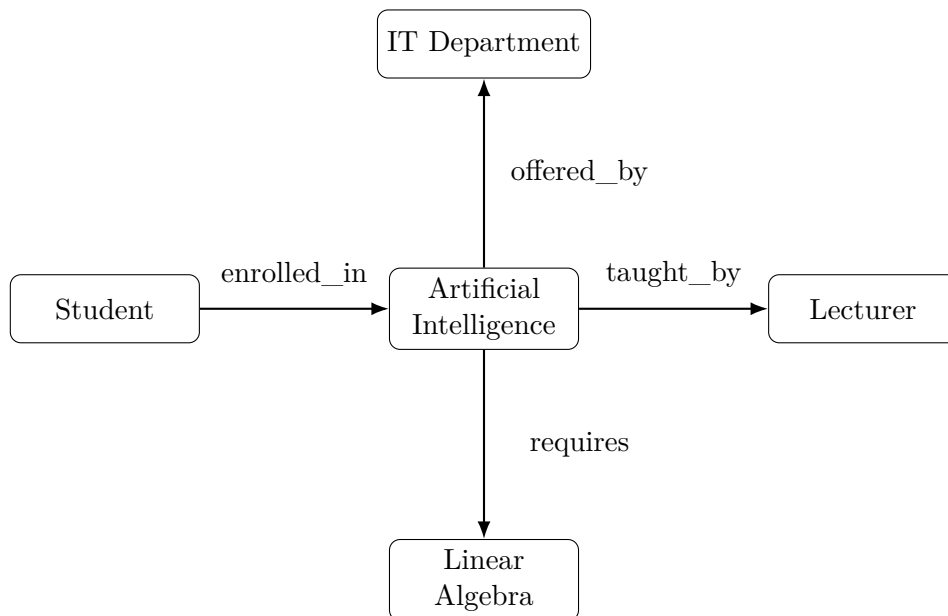
$$(\text{Student}, \text{enrolled_in}, \text{Course}). \quad (15)$$

Another example is

$$(\text{Course}, \text{taught_by}, \text{Lecturer}). \quad (16)$$

7 Example: University Knowledge Graph

Consider the following graph.



A graph makes relationships explicit.

This is particularly useful for questions such as

Which courses require Linear Algebra?

or

Which courses can this student take?

8 Creating a Small Knowledge Graph in Python

A simple graph can first be represented using Python data structures.

```

1 knowledge = [
2     ("AI", "offered_by", "IT Department"),
3     ("AI", "requires", "Linear Algebra"),
4     ("AI", "requires", "Programming"),
5     ("Machine Learning", "requires", "AI"),
6     ("Machine Learning", "taught_by", "Dr. Smith")
7 ]
8
9 for subject, relation, object_ in knowledge:
10     print(subject, relation, object_)

```

The output is conceptually

```

1 AI offered_by IT Department
2 AI requires Linear Algebra
3 AI requires Programming
4 Machine Learning requires AI
5 Machine Learning taught_by Dr. Smith

```

This small example illustrates the fundamental idea of a Knowledge Graph.

9 Graph Databases

For larger systems, graph databases can store entities and relationships explicitly.

One popular graph database is Neo4j.

A graph database allows queries such as

```

1 MATCH (c:Course)-[:REQUIRES]->(p)
2 WHERE p.name = "Linear Algebra"
3 RETURN c

```

The important idea is that the query follows relationships rather than searching only for text similarity.

10 What is RAG?

RAG stands for

Retrieval-Augmented Generation

. (17)

The central idea is simple:

Question → Retrieve Information → Generate Answer. (18)

Instead of asking an LLM to answer using only its internal parameters, the system first retrieves relevant information.

A basic RAG system contains

1. document collection;
2. document splitting;
3. embedding model;

4. vector database;
5. retriever;
6. language model;
7. answer generator.

11 Document Embeddings

Let a text fragment be

$$d_i. \quad (19)$$

An embedding model maps the text into a vector:

$$e_i = f_\theta(d_i) \in \mathbb{R}^m. \quad (20)$$

Similarly, a user query q becomes

$$e_q = f_\theta(q). \quad (21)$$

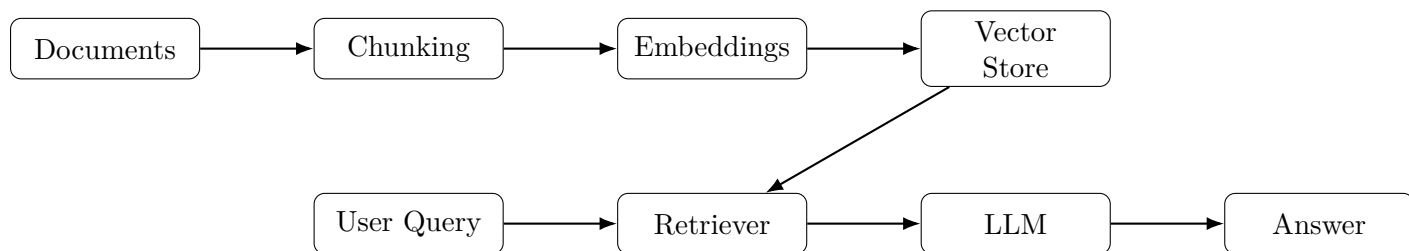
Similarity can be measured using cosine similarity:

$$\text{sim}(e_q, e_i) = \frac{e_q^T e_i}{\|e_q\| \|e_i\|}. \quad (22)$$

The retriever selects the documents with the largest similarity.

12 Basic RAG Pipeline

The complete pipeline is



13 A Minimal RAG Implementation

For teaching, students should first understand the mechanism before using a large framework.

A conceptual implementation is:

```

1 documents = [
2     "Artificial Intelligence requires programming.",
3     "Machine Learning is a branch of Artificial Intelligence.",
4     "Deep Learning uses neural networks.",
5     "RAG retrieves external information before generation."
6 ]
7
8 query = "What is RAG?"
9

```

```

10 # Step 1: embed query
11 query_vector = embed(query)
12
13 # Step 2: embed documents
14 document_vectors = [
15     embed(doc) for doc in documents
16 ]
17
18 # Step 3: compute similarity
19 scores = [
20     cosine_similarity(query_vector, vector)
21     for vector in document_vectors
22 ]
23
24 # Step 4: select relevant documents
25 context = retrieve_top_k(documents, scores, k=2)
26
27 # Step 5: generate answer
28 answer = llm_generate(query, context)
29
30 print(answer)

```

The important conceptual sequence is

$$q \rightarrow e_q \rightarrow \text{similarity} \rightarrow D_k \rightarrow \text{LLM}. \quad (23)$$

14 Why RAG is Important

RAG provides several important advantages.

14.1 Current Information

The knowledge base can be updated without retraining the language model.

14.2 Domain Knowledge

The system can use university documents, company documents, technical manuals, regulations, research papers, or personal notes.

14.3 Evidence

The system can return the retrieved documents as evidence.

14.4 Reduced Hallucination

RAG does not guarantee correctness, but grounding generation in retrieved evidence can reduce unsupported answers.

15 Knowledge Graph versus Vector RAG

Knowledge Graph retrieval and vector retrieval answer different types of questions.

	Knowledge Graph	Vector RAG
Representation	Entities and relations	Text embeddings
Strength	Explicit relationships	Semantic similarity
Typical question	“Who teaches this course?”	“What does this course cover?”
Retrieval	Graph traversal	Vector similarity
Reasoning	Relationship based	Context based
Data	Structured	Unstructured or semi-structured

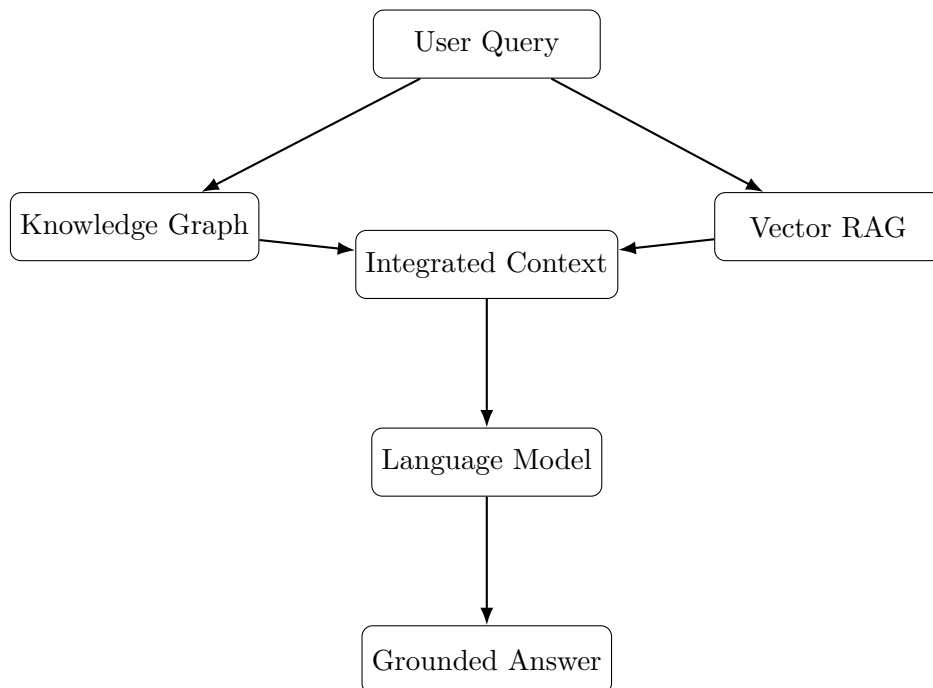
The important conclusion is:

KG and RAG should often be combined rather than compared.

 (24)

16 Knowledge Graph + RAG

A more powerful architecture combines both approaches.



The Knowledge Graph supplies structured relationships while RAG supplies relevant textual evidence.

17 Example: University Intelligent Assistant

Suppose a student asks:

“Can I take Machine Learning next semester?”

The intelligent system may perform the following operations.

17.1 Step 1: Query the Knowledge Graph

The KG contains

$$\begin{aligned} &\text{Machine Learning} \rightarrow \text{requires} \rightarrow \text{Artificial Intelligence}, \\ &\text{Artificial Intelligence} \rightarrow \text{requires} \rightarrow \text{Programming}. \end{aligned} \quad (25)$$

The system checks whether the student has completed the prerequisites.

17.2 Step 2: Retrieve Documents

The RAG subsystem searches course regulations and retrieves relevant passages.

17.3 Step 3: Combine Evidence

The context may contain

```

1 Knowledge Graph:
2 Student -> completed -> Artificial Intelligence
3
4 Document:
5 Machine Learning requires Artificial Intelligence.
```

17.4 Step 4: Generate Answer

The LLM converts the structured evidence and retrieved document into a natural-language response.

This is more powerful than ordinary keyword search.

18 GraphRAG

GraphRAG extends the RAG idea by using graph structure during retrieval.

Conceptually,

$$\text{Question} \rightarrow \text{Graph Retrieval} \rightarrow \text{Context} \rightarrow \text{Generation}. \quad (26)$$

A practical GraphRAG pipeline may include

1. document loading;
2. text splitting;
3. entity extraction;
4. relationship extraction;
5. graph construction;
6. embedding generation;
7. graph/vector retrieval;
8. context construction;
9. LLM generation.

Modern Neo4j tooling supports knowledge graph construction from PDF/text and several retrieval approaches including vector, graph-based, hybrid, and Text-to-Cypher retrieval. [1]

19 GraphRAG Example

Suppose a collection contains research papers about Deep Learning.

The graph may contain

$$\begin{aligned} \text{CNN} &\rightarrow \text{used_for} \rightarrow \text{Image Classification}, \\ \text{LSTM} &\rightarrow \text{used_for} \rightarrow \text{Time Series}, \\ \text{Transformer} &\rightarrow \text{used_for} \rightarrow \text{Language Modeling}. \end{aligned} \quad (27)$$

A user asks:

“Which models are useful for sequential data and why?”

Vector retrieval may find passages containing the words “sequence”, “time series”, and “Transformer”.

Graph retrieval can additionally follow relationships such as

$$\text{Model} \rightarrow \text{used_for} \rightarrow \text{Task}. \quad (28)$$

The two sources of evidence can then be combined.

20 From RAG to Intelligent Agents

RAG answers the question

“What information is relevant?”

An agent must answer a more general question:

“What should I do to achieve the user’s goal?”

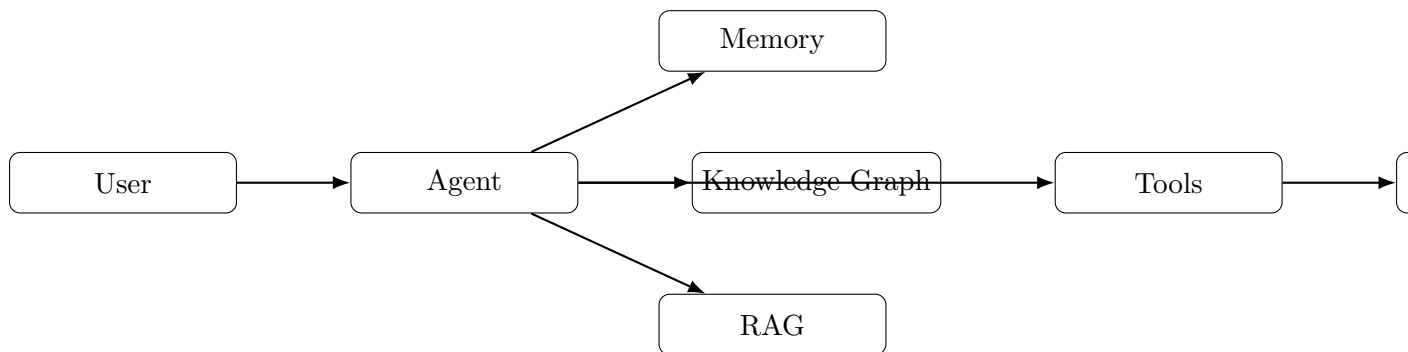
An agent can be modeled conceptually as

$$\text{Agent} = \text{Reasoning} + \text{Memory} + \text{Tools} + \text{Knowledge} + \text{Planning}. \quad (29)$$

A simplified agent loop is

$$\boxed{\text{Observe} \rightarrow \text{Reason} \rightarrow \text{Plan} \rightarrow \text{Act} \rightarrow \text{Observe}.} \quad (30)$$

21 Agent Architecture



The agent therefore becomes the coordinator of the intelligent system.

22 Tools

A tool is an external capability that the agent can invoke.

Examples include:

- calculator;
- Python interpreter;
- database query;
- web search;
- Knowledge Graph query;
- RAG retrieval;
- machine learning model;
- file reader;
- email service;
- scheduling system.

For example, if the user asks

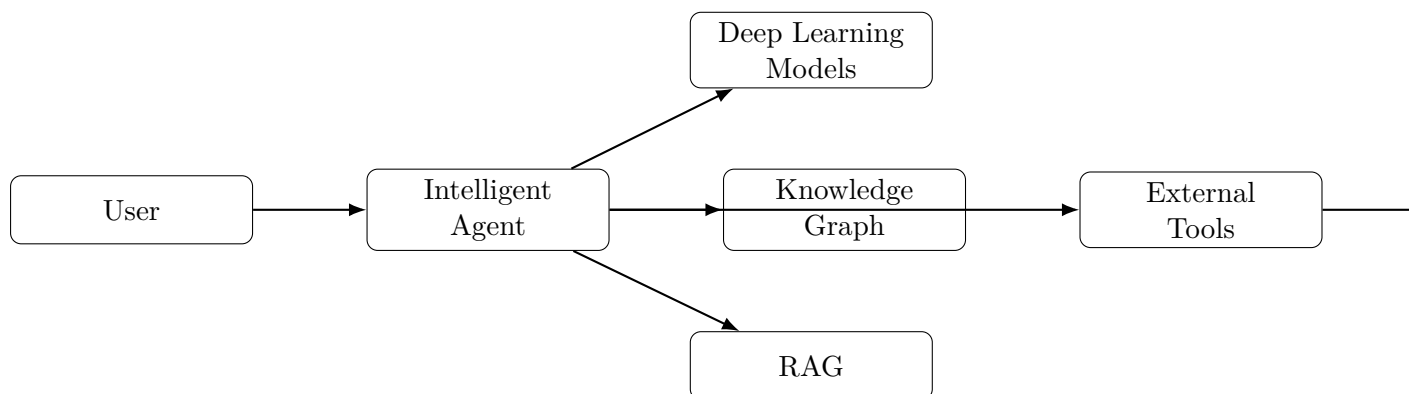
“Predict tomorrow’s stock price and explain the result using recent company information.”

the agent may need to call:

1. a market-data tool;
2. a Deep Learning prediction model;
3. a document retrieval system;
4. a Knowledge Graph;
5. an LLM for explanation.

23 Deep Learning + KG + RAG + Agent

We can now formulate the complete architecture.



This is the central architecture of the lecture.

24 A Unified Mathematical View

Let

$$x \tag{31}$$

denote the user's input.

A learned model produces a representation

$$h = f_{\theta}(x). \tag{32}$$

The query is transformed into a retrieval representation

$$q = f_{\phi}(x). \tag{33}$$

The retrieval system returns

$$D_k = \mathcal{R}(q, \mathcal{K}), \tag{34}$$

where $\mathcal{K}$ is the external knowledge base.

A Knowledge Graph retrieval operator may be written as

$$G_k = \mathcal{G}(q, G), \tag{35}$$

where G is the graph.

The complete context is

$$C = \text{Combine}(D_k, G_k, h). \tag{36}$$

The language model then produces

$$y = \text{LLM}_{\theta}(x, C). \tag{37}$$

For an agent, the process is iterative:

$$s_{t+1} = F(s_t, x_t, C_t, a_t), \tag{38}$$

where

- s_t is the agent state,
- x_t is the current observation,
- C_t is retrieved context,
- a_t is an action.

25 Practical Development Roadmap

Students should not immediately build the complete system.

A progressive development strategy is recommended.

Level 1: Deep Learning

Build one useful model.

Examples:

- image classifier;
- disease classifier;
- stock predictor;
- sentiment classifier;
- time-series predictor.

Level 2: Semantic Search

Convert documents into embeddings.

$$d_i \rightarrow e_i. \quad (39)$$

Then implement similarity search.

Level 3: Basic RAG

Add an LLM:

$$q \rightarrow \text{retrieve}(q) \rightarrow \text{LLM}. \quad (40)$$

Level 4: Knowledge Graph

Create

$$G = (V, E). \quad (41)$$

Implement graph queries.

Level 5: Hybrid RAG

Combine

$$\text{Vector Retrieval} + \text{Graph Retrieval}. \quad (42)$$

Level 6: Agent

Add:

$$\text{Memory} + \text{Tools} + \text{Planning} + \text{Action}. \quad (43)$$

26 Practical Project

The final student project should be:

Personal / Domain Intelligent Assistant

Each group selects a domain.

Examples include:

- university assistant;
- medical information assistant;
- financial assistant;
- e-commerce assistant;
- tourism assistant;
- research assistant;
- programming assistant;
- agricultural assistant.

The project should contain at least four components:

1. one Deep Learning model;
2. one Knowledge Graph;
3. one RAG pipeline;
4. one agent or tool-use component.

27 Example Project: University Assistant

The knowledge base may contain:

- course information;
- lecturer information;
- curriculum;
- prerequisites;
- university regulations;
- student information;
- lecture documents.

The Knowledge Graph contains structured information:

$$\begin{aligned}
 &Student \rightarrow Course, \\
 &Course \rightarrow Lecturer, \\
 &Course \rightarrow Prerequisite, \\
 &Course \rightarrow Department.
 \end{aligned}
 \tag{44}$$

RAG contains unstructured information:

$$PDFs \rightarrow Chunks \rightarrow Embeddings \rightarrow Vector Store. \tag{45}$$

The agent coordinates the system.

28 Example User Interaction

User:

I have completed Programming and Linear Algebra. What should I study next?

The agent may perform:

1. query student records;
2. query the Knowledge Graph;
3. identify eligible courses;
4. retrieve course descriptions;
5. rank the courses;
6. explain the recommendations;
7. provide document evidence.

The final response could be:

Based on your completed courses, Artificial Intelligence and Machine Learning are possible next courses. Machine Learning requires Artificial Intelligence according to the curriculum, so Artificial Intelligence is recommended first.

The answer is therefore generated from

$$\boxed{\text{Student Data} + \text{Knowledge Graph} + \text{RAG} + \text{LLM}.} \tag{46}$$

29 Evaluation of an Intelligent System

Students should not evaluate an intelligent system only by asking whether the answer “looks good.”

Several dimensions should be evaluated.

29.1 Prediction Accuracy

For a Deep Learning component:

$$Accuracy = \frac{\text{correct predictions}}{\text{total predictions}}. \quad (47)$$

Other metrics include

- precision;
- recall;
- F1-score;
- MAE;
- RMSE;
- AUC.

29.2 Retrieval Quality

Measure whether relevant documents are retrieved.

For example:

$$Recall@k = \frac{\text{relevant retrieved items}}{\text{all relevant items}}. \quad (48)$$

29.3 Answer Quality

Evaluate:

- correctness;
- relevance;
- completeness;
- groundedness;
- citation/evidence quality.

29.4 Agent Performance

Evaluate whether the agent:

- selects the correct tool;
- retrieves appropriate information;
- follows the correct workflow;
- avoids unnecessary actions;
- completes the user's goal.

30 Common Failure Modes

Students should understand that intelligent systems can fail.

30.1 Deep Learning Failure

$$\text{Poor Data} \rightarrow \text{Poor Model.} \quad (49)$$

Typical problems include:

- overfitting;
- data leakage;
- class imbalance;
- distribution shift.

30.2 Knowledge Graph Failure

The graph may contain:

- missing entities;
- incorrect relationships;
- duplicate entities;
- outdated information.

30.3 RAG Failure

The retriever may return irrelevant documents.

Then

$$\text{Bad Retrieval} \rightarrow \text{Bad Context} \rightarrow \text{Bad Answer.} \quad (50)$$

30.4 LLM Failure

The model may generate an unsupported answer.

Therefore,

$$\text{LLM Output} \neq \text{Guaranteed Truth.} \quad (51)$$

30.5 Agent Failure

An agent may select the wrong tool or execute an incorrect sequence of actions.

This motivates systematic evaluation.

31 Implementation Environment

A practical student environment is:

Component	Recommended technology
Programming	Python
Development	Jupyter Notebook / Spyder
Deep Learning	PyTorch
Embeddings	Sentence Transformers or API embeddings
Vector search	FAISS / Chroma / vector database
Knowledge Graph	Neo4j
Graph query	Cypher
RAG framework	LangChain or direct Python
GraphRAG	Neo4j GraphRAG Python package
LLM	API model or local model
Visualization	Matplotlib / NetworkX

Neo4j currently provides an official Python GraphRAG package that supports knowledge-graph construction and retrieval workflows, including vector, graph, and hybrid retrieval. [1]

LangChain documentation also provides practical workflows for semantic search over PDFs and RAG applications. [2]

32 Suggested Student Workflow

A student should implement the project in the following order.

1. Select a domain.
2. Collect data.
3. Build a Deep Learning model.
4. Collect domain documents.
5. Split documents into chunks.
6. Generate embeddings.
7. Build a vector index.
8. Implement semantic retrieval.
9. Implement basic RAG.
10. Identify entities and relationships.
11. Build a Knowledge Graph.
12. Implement graph queries.
13. Combine graph retrieval and vector retrieval.
14. Add an LLM.
15. Add tools.
16. Add memory.
17. Implement an agent loop.
18. Evaluate the complete system.

33 Mini Project 1: Semantic Search

Objective:

Build a search engine over a collection of documents.

Input:

- 20–100 documents;
- one query.

Output:

Return the top- k relevant passages.

Students should report:

- embedding model;
- vector dimension;
- similarity function;
- k ;
- retrieval examples.

34 Mini Project 2: RAG Chatbot

Build

$$\text{Documents} \rightarrow \text{RAG} \rightarrow \text{Chatbot.} \quad (52)$$

The chatbot must answer questions using the provided documents.

Each answer should contain:

- answer;
- retrieved evidence;
- document identifier;
- optional page number.

35 Mini Project 3: Knowledge Graph

Students construct a graph containing at least

- 50 entities;
- 100 relationships;
- 5 relationship types;
- 3 entity types.

They should implement at least five Cypher queries.

Example:

```
1 MATCH (c:Course)-[:REQUIRES]->(p:Course)
2 RETURN c.name, p.name
```

36 Final Project: Intelligent Agent

The final system should have the following architecture:

$$\text{User} \rightarrow \text{Agent} \rightarrow \left[\begin{array}{c} \text{Deep Learning} \\ \text{Knowledge Graph} \\ \text{RAG} \\ \text{Tools} \\ \text{Memory} \end{array} \right] \rightarrow \text{Action.} \quad (53)$$

The student should demonstrate at least five user queries.

For every query, the report should show:

1. user question;
2. agent reasoning workflow;
3. selected tools;
4. retrieved KG information;
5. retrieved documents;
6. final answer;
7. evidence;
8. evaluation.

37 Recommended Final Report Structure

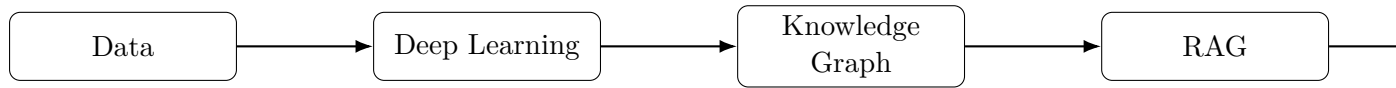
The final project report should contain:

1. Introduction
2. Problem Definition
3. Data Collection
4. Deep Learning Model
5. Knowledge Graph Design
6. RAG Architecture
7. Agent Architecture
8. System Implementation
9. Experimental Evaluation
10. Error Analysis
11. Limitations
12. Conclusion

The report should include both architecture diagrams and experimental evidence.

38 The Big Picture

The development path of this course can now be summarized as



The progression is not simply a sequence of technologies.
It represents an evolution in system capability:

$$\begin{aligned}
 &\text{Deep Learning} \rightarrow \text{learn patterns,} \\
 &\text{Knowledge Graph} \rightarrow \text{represent relationships,} \\
 &\text{RAG} \rightarrow \text{retrieve information,} \\
 &\text{LLM} \rightarrow \text{understand and generate language,} \\
 &\text{Agent} \rightarrow \text{reason, plan and act.}
 \end{aligned} \tag{54}$$

Therefore,

$$\boxed{\text{Intelligent System} = \text{Deep Learning} + \text{Knowledge Graph} + \text{RAG} + \text{LLM} + \text{Agent}.} \tag{55}$$

39 Exercises

Exercise 1

Explain the difference between a Deep Learning model and a Knowledge Graph.

Exercise 2

Why can a trained neural network fail to answer questions about new information?

Exercise 3

Given

$$(\text{Course}, \text{requires}, \text{Programming}), \tag{56}$$

identify the subject, relation, and object.

Exercise 4

Explain why vector similarity is useful for RAG.

Exercise 5

Design a Knowledge Graph for an e-commerce system.

Use at least:

- Product;
- Customer;
- Order;
- Category;
- Seller.

Exercise 6

Design a RAG system for university regulations.

Exercise 7

Explain the difference between ordinary RAG and GraphRAG.

Exercise 8

Design an agent that can answer:

“Find a suitable product, explain why it is suitable, and compare it with two alternatives.”

Identify:

- memory;
- tools;
- Knowledge Graph;
- RAG;
- Deep Learning model;
- final action.

40 Final Message to Students

The objective of Intelligent System Development is not simply to learn how to call an LLM.

A modern intelligent system should be able to

$$\boxed{\text{Learn} \rightarrow \text{Know} \rightarrow \text{Retrieve} \rightarrow \text{Reason} \rightarrow \text{Act.}} \quad (57)$$

Deep Learning teaches the machine from data.

Knowledge Graphs organize what the system knows.

RAG gives the system access to relevant external information.

Large Language Models provide powerful language understanding and generation.

Agents combine these capabilities into a system that can pursue a goal.

The final objective of the course is therefore:

Do not build only a model. Build an intelligent system around the model.

References

- [1] Neo4j, *Neo4j GraphRAG for Python*, Neo4j Documentation, 2026.
- [2] LangChain, *Learn: Semantic Search and Retrieval Augmented Generation*, LangChain Documentation, 2026.
- [3] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [4] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [5] T. Mikolov et al., “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.
- [6] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations*, 2017.